



UNIVERSITY OF CYBERJAYA (UOC)

PERSIARAN BESTARI, CYBER 11

63000

CYBERJAYA

SELANGOR DARUL EHSAN

BACHELORS IN CREATIVE MULTIMEDIA

ASSIGNMENT 2

WEB DEVELOPMENT CBMM2413

PERSONAL PORTFOLIO WEBSITE – FRONT END DEVELOPMENT

[Bennybenportfolio.netlify.app](https://bennybenportfolio.netlify.app)

A tactical cyber-aesthetic showcase & creative director interface

METADATA

Version 1.0.0 | developer & designer portfolio | production-ready

STUDENT'S NAME	MATRIX NO
ATHAAVAN A/L V.P.GANISEN	2503-0911

Submitted to

Dr Safwan Mahmood Othman Al-Selwi

Lecturer for

Web Development

Submission Date: 17/5/2026

TABLE OF CONTENTS

1. EXECUTIVE SUMMARY & CORE CONCEPT	3
2. CORE FEATURES & INTERFACE ELEMENTS	4
3. TECH STACK & ARCHITECTURE	5
4. ENGINEERING CHALLENGES & OPTIMIZATIONS.....	6
5. VISUAL ASSET	7
6. DEPLOYMENT PIPELINE & LOCAL ENVIRONMENT	9
7. DEVELOPER LOGS: OVERCOMING TECHNICAL FRICTION	11
8. MAINTENANCE PROTOCOL: PUSHING UPDATES VIA VS CODE	12

1. EXECUTIVE SUMMARY & CORE CONCEPT

This project is a highly customized digital portfolio engineered to serve as the definitive intersection of technical UI/UX development and comprehensive brand identity design. Moving away from conventional, sterile minimalist templates, the interface embraces a distinct Gamer / Creative Director aesthetic. It is built to immediately communicate a specialized design philosophy: one that merges the structural, functional discipline of Bauhaus design with the high-contrast, immersive visual language of Cyberpunk.

Rather than functioning as a standard static gallery, the environment is constructed as a tactical digital dossier. It utilizes sharp UI grids, rich textures, and precision glassmorphism that mimics advanced terminal interfaces. Every element from the meticulous typography to the dynamic scanner line overlays is intentionally coded to reflect a deep understanding of immersive, cinematic motion graphics. It is a living proof of concept demonstrating how complex vector assets, logo frameworks, and cohesive branding mockups can be elevated through an interactive, heavily stylized web environment.

The primary objective was to build a performant, ultra-responsive web experience that commands attention. It is optimized specifically to house high energy promotional campaigns, environmental design graphics, and detailed technical analyses without the visual constraints of third party portfolio platforms. By seamlessly bridging raw front end web architecture with advanced graphic design capabilities, this portfolio stands as a statement of absolute creative control, designed to engage clients and collaborators in a fully realized digital atmosphere.

2. CORE FEATURES & INTERFACE ELEMENTS

Tactical Cyber-Aesthetic UI: Designed with high contrast elements, custom borders, and a sophisticated glassmorphism styling that mimics advanced heads up displays (HUDs).

Immersive Scanner Line Overlays: A subtle, dynamic background scanline effect that reinforces the high tech, digital terminal theme without compromising readability.

Dynamic Asset Loading Engine: Built with intelligent JavaScript logic to automatically parse, categorize, and inject project images dynamically from dedicated media folders, simplifying future portfolio updates.

Cinematic Motion Design: Features premium, fluid fade in animations and smooth scrolling mechanics that create a cohesive, polished user flow from header to footer.

Dynamic Background Texturing: Programmatically injected textures that ensure background graphics load smoothly and responsively without lagging the client side rendering.

3. TECH STACK & ARCHITECTURE

To maintain absolute control over performance, layout precision, and animation keyframes, the project bypasses heavy monolithic frameworks in favour of a clean, vanilla architecture.

Development Stack

Layer	Technology Used	Implementation Detail
Structure	HTML5	Semantic structure optimized for SEO and modern web scrapers.
Styling & Layout	CSS3 / Advanced Grid	Leverages CSS Grid for complex, asymmetrical tactical panels and glassmorphic layouts.
Interactivity	Vanilla JavaScript	Powers the dynamic asset folder injection, animation triggers, and scroll behaviors.
Hosting & CI/CD	Netlify	Edge-network hosting with automated deployment pipelines.

Clean File Structure

The codebase follows a strict separation of concerns to ensure clean scalability:

Plaintext

```
|— index.html      # Main entry point and semantic structure
|— css/
|   |— style.css   # Layouts, tactical grids, glassmorphism, and scanlines
|— js/
|   |— main.js     # Folder automation, dynamic textures, and animations
|— assets/
|   |— UI/         # HUD elements, overlays, and background textures
|   |— projects/   # Categorized folders for dynamic loading
```

4. ENGINEERING CHALLENGES & OPTIMIZATIONS

Mobile Responsive Overflow Correction

The Issue: Early development builds suffered from minor horizontal scrolling/overflow on mobile viewports due to the rigid nature of wide tactical layout panels and absolute-positioned scanner overlays.

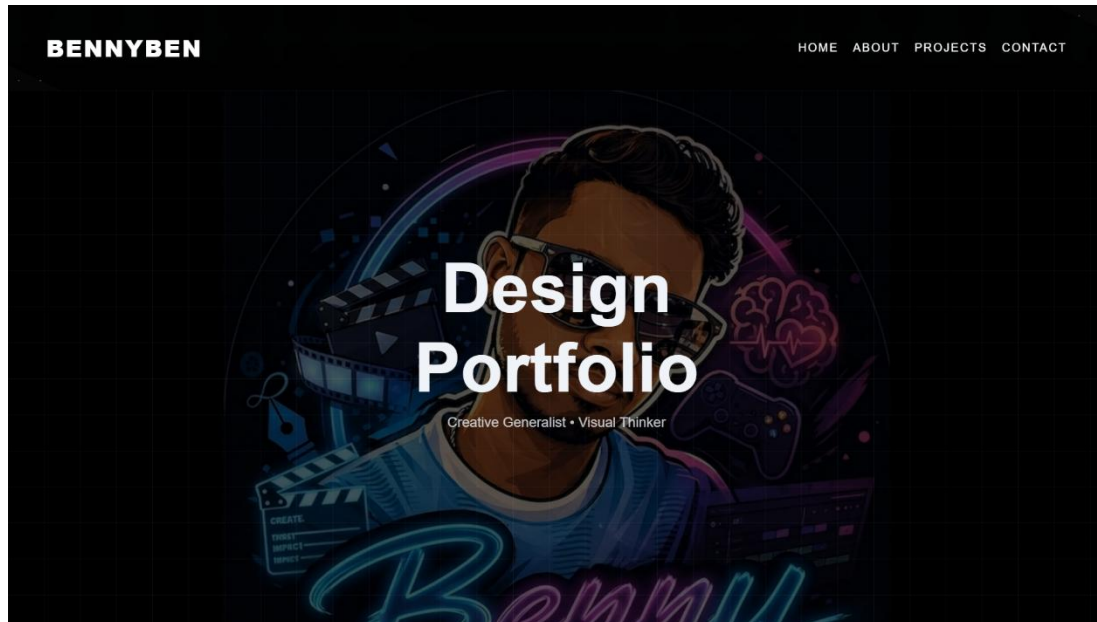
The Fix: Implemented defensive CSS layouts using max-width: 100%, refined flex-wrap rules, and utilized overflow-x: hidden selectively on container wrappers to lock down the viewport. The entire UI now seamlessly scales down from ultra-wide monitors to mobile screens while preserving its dense, tactical aesthetic.

Dynamic Layout Asset Management

Rather than hardcoding dozens of portfolio images into the HTML file, the JS layer automatically handles asset injection, keeping the codebase incredibly modular.

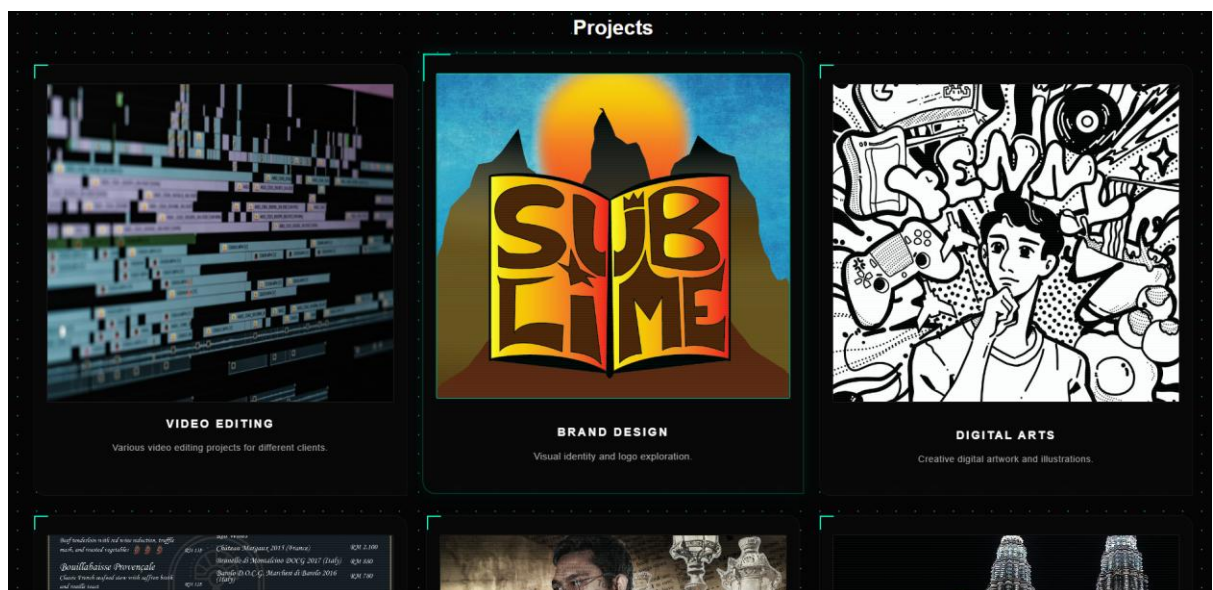
5. VISUAL ASSET

Screenshot 1: The Main Interface (Full Page View)



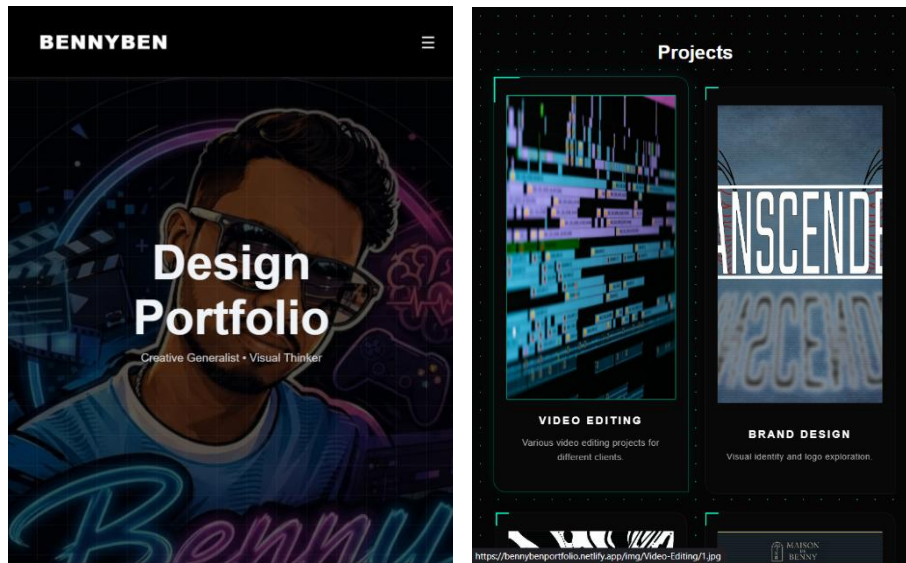
Caption: Figure 1.1: Desktop view highlighting the CSS Grid tactical layouts, glassmorphism panel styling, and active scanner line overlays.

Screenshot 2: Dynamic Project Showcase (Grid Zoom-In)



Caption: Figure 1.2: Close-up of the dynamic asset grid populating items dynamically via JavaScript.

Screenshot 3: Mobile Viewport Performance



Caption: *Figure 1.3: Responsive view showing zero horizontal overflow and flawless scaling of tactical border details on mobile displays.*

6. DEPLOYMENT PIPELINE & LOCAL ENVIRONMENT

This project utilizes a modern, lightweight CI/CD pipeline, moving from local development directly to edge-network hosting without the need for manual server configuration.

Step-by-Step Setup

Local Initialization (VS Code) To run the environment locally for debugging or testing new assets:

Bash

Clone the repository to your local machine

```
git clone https://github.com/your-username/bennyben-portfolio.git
```

Navigate into the project directory

```
cd bennyben-portfolio
```

Open the workspace in Visual Studio Code

```
code .
```

Note: Because this project uses vanilla web technologies and dynamic JavaScript fetching, it must be run on a local server to bypass CORS restrictions. Use the VS Code **Live Server** extension to launch the site on <http://localhost:5500>.

Version Control (GitHub) All source code is tracked via Git. To push updates to the repository:

Bash

```
git add .
```

```
git commit -m "UI: BennyBenPortfolio"
```

```
git push origin main
```

Production Deployment (Netlify) The live application is hosted on Netlify, which is directly hooked into the GitHub repository.

Continuous Deployment: Every push to the main branch triggers an automatic build and deployment sequence.

Edge Routing: Netlify serves the static assets globally, ensuring the heavy visual textures and CSS animations load instantly regardless of the user's region.

7. DEVELOPER LOGS: OVERCOMING TECHNICAL FRICTION

Building a highly customized, visually dense interface without relying on heavy frameworks introduced specific engineering challenges. Here are the core friction points encountered during development and how they were neutralized.

Friction Point Alpha: The Static Host / Asset Path Trap

The Struggle: During local development on Windows/Mac, image paths are typically case insensitive. However, Netlify uses strict Linux based servers. Pushing code with a path like `/assets/UI/logo.PNG` while the file was actually `logo.png` caused broken images and missing scanner lines in the live production build, despite working perfectly on localhost.

The Resolution: Audited the entire file structure to enforce a strict lowercase naming convention for all static assets and updated the DOM injection scripts to match.

Friction Point Beta: Glassmorphism Render Lag on Mobile

The Struggle: The tactical cyber-aesthetic relies heavily on CSS backdrop filter: `blur()` to create the frosted glass effect on the HUD panels. When combined with the dynamic scanner line overlays, mobile devices experienced significant frame drops and scrolling lag.

The Resolution: Hardware acceleration was forced on critical UI components. By adding `will change: transform`, backdrop filter and utilizing `transform: translateZ(0)` on the heaviest containers, rendering was offloaded to the device's GPU, restoring a smooth 60fps scroll.

Friction Point Gamma: Dynamic Folder Parsing

The Struggle: Because the site is statically hosted on Netlify (without a Node.js or Python backend), using JavaScript to automatically read the contents of a local `/projects` folder to generate the portfolio grid was impossible due to browser security restrictions.

The Resolution: Engineered a lightweight JSON mapping system. Instead of the browser trying to read a server directory, the JavaScript engine fetches a local `data.json` file containing the precise metadata, tags, and image paths, injecting the HTML nodes dynamically into the CSS Grid. This mimics database behavior while maintaining the speed of a static site.

8. MAINTENANCE PROTOCOL: PUSHING UPDATES VIA VS CODE

Because the Netlify deployment is directly tethered to the GitHub repository, pushing updates to the live site requires zero manual server uploads. Once the initial setup is complete, all future asset updates, CSS tweaks, or bug fixes are handled directly through Visual Studio Code.

The Standard Update Cycle

Whenever you add a new project image to your /projects folder or modify the interface, follow this strict 3-step Git protocol using the VS Code integrated terminal (Ctrl + `).

Step 1: Stage the Modifications Once your local changes are saved and tested on your Live Server, tell Git to track the new or modified files.

Bash

Stages every changed file in your workspace

git add .

Pro-Tip: If you only want to push a specific file (e.g., just the stylesheet), you can target it directly: *git add css/style.css*.

Step 2: Commit with a Status Log Lock in the changes with a clear, descriptive message. This acts as your project's changelog, making it easy to roll back if an update breaks the live UI.

Bash

Example commit messages for your workflow:

git commit -m "ASSETS: Added WingStar branding mockups to project grid"

or

git commit -m "UI: Tweaked opacity on the glassmorphism scanner lines"

Step 3: Push to the Live Edge-Network Execute the push command to send your local commits up to the GitHub repository.

Bash

Pushes the update to your main production branch

```
git push origin main
```

Automated Live Deployment (The Netlify Hook)

Once the `git push origin main` command executes successfully in VS Code:

GitHub registers the new commit.

A webhook automatically pings Netlify.

Netlify rebuilds the site on their servers and deploys the new version globally within seconds.

You do not need to log into Netlify to see your changes; simply refresh `bennybenportfolio.netlify.app` after pushing your code.

Procedure To Update The Website After Changes

Run ONLY These 3 Commands

```
git add .
```

```
git commit -m "Updated website"
```

```
git push
```

Your Final Workflow

Edit in VS Code



```
git add .
```

```
git commit -m "Update"
```

```
git push
```



GitHub updates



Netlify auto deploys



Live website updates

That's the real-world workflow used by developers and designers.